# DINE SYNC

QR-Based Smart Restaurant Ordering Platform

**Comprehensive Technical Documentation**
Backend Microservices Architecture & Scope of Work

Prepared by: Rupankar Das
Version 1.0 | May 2026

# 1. Executive Summary

DineSync is a multi-tenant SaaS platform designed for restaurants, cafés, bars, and food courts to digitize their dine-in ordering experience using QR codes. Each table has a unique QR code that directs customers to a live, interactive digital menu specific to their table and restaurant.

Customers can browse the menu, customize orders, place them directly, and pay — all from their mobile browser with no app download required. Orders flow in real time to the kitchen queue and floor manager dashboard, powered by WebSockets and message queues.

**Project Goal**

Build a production-grade, scalable backend using a microservices architecture that powers a multi-tenant restaurant SaaS platform — deployable on AWS with Docker and CI/CD pipelines.

# 2. System Overview

## 2.1 Target Audience

| Audience | Use Case |
|---|---|
| Restaurants | Full-service dine-in ordering with chef queue management |
| Cafés | Quick menu display, ordering, and payment at table |
| Bars | Tab management, item availability, and fast ordering |
| Food Courts | Multi-stall menu display with QR-based table assignment |

## 2.2 High-Level Architecture

The system follows a microservices architecture where each domain concern is an independent service with its own database. Services communicate via REST (synchronous) and a message broker (asynchronous). All external traffic enters through a central API Gateway.

**Architecture Pattern**

Multi-tenant microservices with API Gateway → Individual Services → PostgreSQL (per service) + Redis (shared cache/queue) + Kafka (event streaming)

# 3. Microservices Breakdown

The platform is composed of 9 independent backend services, each responsible for a single domain:

## 3.1 API Gateway Service

The single entry point for all client requests. Responsible for routing, rate limiting, and authentication validation.

- Route requests to correct downstream microservice
- Validate JWT tokens before forwarding
- Rate limiting per tenant and per IP
- Request/response logging and tracing
- Subdomain-based tenant resolution (e.g., pizza-hut.dinesync.com)

| Technology | Purpose |
|---|---|
| Nginx / Express Gateway | Reverse proxy and routing |
| JWT Middleware | Token validation |
| Redis | Rate limit counters |

## 3.2 Auth Service

Handles all authentication and authorization across the platform. Issues JWTs and manages role-based access.

- User registration and login (email + password)
- JWT access token + refresh token flow
- OAuth2 support for Google Sign-In
- Role management: Customer, Floor Manager, Chef, Owner
- Password reset and session invalidation

| Role | Permissions |
|---|---|
| Customer | View menu, place order, pay, view own order status |
| Floor Manager | View live orders, approve/reject orders, manage tables |
| Chef | View cooking queue, update item status (In Progress / Ready) |
| Owner | Full access: analytics, menu, stock, staff, settings |

## 3.3 Restaurant (Tenant) Service

Manages multi-tenancy. Each restaurant is an isolated tenant with its own subdomain, configuration, and subscription plan.

- Restaurant onboarding and profile management
- Subdomain provisioning and routing metadata
- Subscription plan management (Free / Starter / Growth / Enterprise)
- Table creation and unique QR code generation per table
- Staff account creation under a restaurant tenant

## 3.4 Menu Service

Manages everything related to the restaurant's menu — categories, dishes, pricing, customization options, and availability.

- CRUD for menu categories and dishes
- Dish customization options (e.g., spice level, add-ons, portions)
- Real-time availability toggle (in stock / out of stock)
- Menu update limits enforced per subscription tier
- Image upload for dishes via AWS S3
- Auto-hide dishes when inventory falls to zero

## 3.5 Order Service

The core transactional service. Handles order placement, tracking, and lifecycle management.

- Accept orders from QR-scanned table sessions
- Order approval flow: Customer places → Manager approves → Chef queue
- Order status lifecycle: Placed → Approved → In Progress → Ready → Served
- Publish order events to Kafka for real-time propagation
- Order history per table session and per restaurant

| Order Status | Triggered By | Notified To |
|---|---|---|
| Placed | Customer | Floor Manager |
| Approved | Floor Manager | Chef Queue |
| In Progress | Chef | Floor Manager, Customer |
| Ready | Chef | Floor Manager, Customer |
| Served | Floor Manager | Customer (bill prompt) |

## 3.6 Kitchen Queue Service

Manages the real-time cooking queue for chefs. Orders appear in the queue after manager approval and are ordered by timestamp or priority.

- Consume order events from Kafka
- Maintain per-restaurant cooking queue in Redis
- Chefs mark items as In Progress or Ready
- Priority queue support (e.g., VIP table, urgent orders)
- Display estimated wait time per item

### 3.7 Payment Service

Handles payment initiation, verification, and receipt generation. Integrates with Razorpay, Stripe, and UPI.

- Create payment order via Razorpay/Stripe
- Webhook listener for payment confirmation
- Payment directly to restaurant's linked business account
- E-bill/invoice generation per table session (PDF via S3)
- Refund handling for cancelled orders

### 3.8 Inventory & Stock Service

Tracks dish availability in real time and notifies owners when items fall below threshold.

- Track stock count per dish
- Auto-update menu availability when stock hits zero
- Threshold alerts: notify owner via email/notification when low
- Sync with Order Service to decrement stock on order placement
- Restock updates from Owner dashboard

### 3.9 Notification Service

Handles all real-time and async notifications across the platform.

- WebSocket push to Customers, Floor Manager, and Chef dashboards
- Email notifications for owner alerts (low stock, daily summary)
- WhatsApp order alerts (Growth plan and above)
- Consumes events from Kafka to trigger appropriate notifications

## 4. Complete Project Flow

### 4.1 Customer Journey

| Step | Action | Service Involved |
|---|---|---|
| 1 | Customer scans table QR code | Restaurant Service (resolve tenant + table) |
| 2 | Menu page loads for that table | Menu Service (fetch dishes + availability) |
| 3 | Customer customizes and adds items to cart | Frontend only (client-side state) |
| 4 | Customer places order | Order Service (create order, publish to Kafka) |
| 5 | Floor Manager approves order | Order Service + Notification Service |
| 6 | Order enters chef queue | Kitchen Queue Service (consume Kafka event) |
| 7 | Chef marks items In Progress / Ready | Kitchen Queue Service + Notification Service |
| 8 | Customer is notified order is ready | Notification Service (WebSocket) |
| 9 | Customer requests bill | Order Service + Payment Service |
| 10 | Customer pays via Razorpay/UPI | Payment Service (webhook confirms) |
| 11 | E-bill generated and sent | Payment Service (PDF to S3, email to customer) |

## 4.2 Data Flow Diagram (Textual)

**Request Flow**

Client (Mobile Browser) → API Gateway (auth + routing) → Auth Service / Menu Service / Order Service → Kafka (order.placed event) → Kitchen Queue Service (chef queue update) → Notification Service (WebSocket push to all dashboards) → Redis (real-time queue state) → PostgreSQL (persistent storage per service)

## 4.3 Restaurant Onboarding Flow

- Owner registers on DineSync and selects a subscription plan
- Restaurant profile created — subdomain assigned (e.g., myrestaurant.dinesync.com)
- Owner creates tables and downloads unique QR codes per table
- Owner creates staff accounts (Floor Managers, Chefs) with role assignment
- Owner builds menu: categories, dishes, prices, images, customizations
- Restaurant goes live — customers can scan and order immediately

# 5. Database Design

## 5.1 Database Strategy

Each microservice owns its own PostgreSQL database (Database-per-Service pattern). No service queries another service's database directly — all cross-service data is accessed via API calls or Kafka events.

| Service | Database | Key Tables |
|---|---|---|
| Auth Service | auth_db (PostgreSQL) | users, roles, refresh_tokens, sessions |
| Restaurant Service | restaurant_db (PostgreSQL) | restaurants, tables, qr_codes, subscriptions, staff |
| Menu Service | menu_db (PostgreSQL) | categories, dishes, customizations, dish_images |
| Order Service | order_db (PostgreSQL) | orders, order_items, order_status_logs |
| Kitchen Queue Service | Redis | queue:{restaurant_id} (sorted set by timestamp) |
| Payment Service | payment_db (PostgreSQL) | payments, invoices, refunds |
| Inventory Service | inventory_db (PostgreSQL) | stock_items, stock_alerts, stock_logs |
| Notification Service | Redis | active_socket_sessions, notification_logs |

## 5.2 Key Schema Highlights

### Orders Table

| Column | Type | Description |
|---|---|---|
| id | UUID | Primary key |
| restaurant_id | UUID | Tenant identifier |
| table_id | UUID | Which table placed the order |
| status | ENUM | placed \| approved \| in_progress \| ready \| served \| cancelled |
| total_amount | DECIMAL | Total order value |
| payment_status | ENUM | pending \| paid \| refunded |
| created_at | TIMESTAMP | Order placement time |

### Dishes Table

| Column | Type | Description |
|---|---|---|
| id | UUID | Primary key |
| restaurant_id | UUID | Tenant identifier |

| Column | Type | Description |
|---|---|---|
| category_id | UUID | Menu category |
| name | VARCHAR | Dish name |
| price | DECIMAL | Base price |
| is_available | BOOLEAN | Live availability toggle |
| stock_linked | BOOLEAN | Whether tied to inventory service |

# 6. Technology Stack

| Layer | Technology | Justification |
|---|---|---|
| Runtime | Node.js + Express.js | Familiar stack, non-blocking I/O for high concurrency |
| Language | TypeScript | Type safety across all services |
| Primary DB | PostgreSQL (per service) | ACID compliance for transactional data |
| Cache / Queue | Redis | In-memory speed for chef queues and rate limiting |
| Event Streaming | Apache Kafka | High-throughput async communication between services |
| ORM | Prisma | Type-safe DB access with migration support |
| API Docs | Swagger / OpenAPI | Auto-generated docs per service |
| Auth | JWT + OAuth2 | Stateless auth with role-based middleware |
| Real-time | Socket.io | WebSocket connections for live dashboards |
| File Storage | AWS S3 | Dish images, QR codes, PDF invoices |
| Cloud Infra | AWS EC2 + RDS + ALB | Scalable, production-grade hosting |
| Containerization | Docker + Docker Compose | Consistent local and production environments |
| CI/CD | GitHub Actions | Automated testing, build, and deployment |
| Process Manager | PM2 | Service uptime management on EC2 |
| Payment | Razorpay + Stripe | Indian market (Razorpay) + international (Stripe) |
| Monitoring | AWS CloudWatch | Logs and metrics across all services |

# 7. Scope of Work

## 7.1 Phase 1 — Core Foundation (Weeks 1–4)

**Phase 1 Goal**

Get the fundamental ordering flow working end-to-end. Minimum viable backend that can be demonstrated.

- API Gateway setup with JWT validation and subdomain routing
- Auth Service: registration, login, JWT, RBAC middleware
- Restaurant Service: onboarding, table creation, QR code generation
- Menu Service: CRUD for categories and dishes, availability toggle
- Order Service: place order, status lifecycle, basic approval flow
- Swagger documentation for all Phase 1 services
- Docker Compose setup for local development
- PostgreSQL schemas and Prisma migrations for all services

## 7.2 Phase 2 — Real-time & Payments (Weeks 5–8)

**Phase 2 Goal**

Add real-time capabilities and payment integration — the features that make the platform production-worthy.

- Kitchen Queue Service with Redis sorted sets
- Notification Service with Socket.io WebSocket connections
- Kafka integration: Order Service publishes, Queue and Notification Services consume
- Payment Service: Razorpay integration, webhook handling, invoice generation
- Inventory Service: stock tracking, auto-availability sync with Menu Service
- AWS S3 integration: dish images, QR code storage, PDF invoices
- GitHub Actions CI/CD pipeline with test and deploy stages

## 7.3 Phase 3 — SaaS & Scale (Weeks 9–12)

**Phase 3 Goal**

Add multi-tenancy enforcement, subscription tiers, and production deployment on AWS.

- Subscription plan enforcement: table limits, menu update limits, staff role limits
- Owner analytics dashboard API: revenue, popular dishes, peak hours
- WhatsApp notification integration (Twilio / Meta API)
- AWS deployment: EC2 instances per service, RDS for PostgreSQL, ALB for load balancing

- Multi-tenancy hardening: tenant isolation in all services
- Rate limiting per tenant tier
- Full E2E testing with Postman/Newman test suites
- Performance testing and optimization (Redis caching strategy)

### 7.4 Out of Scope (Backend Only)

| Out of Scope | Reason |
|---|---|
| Frontend / Mobile App | Backend-only project; Swagger docs serve as API interface |
| POS Integration | Enterprise feature — Phase 3+ or separate project |
| White-labeling | Enterprise tier — future roadmap |
| ML-based recommendations | Out of scope for v1 |

## 8. API Design Overview

### 8.1 API Conventions

- Base URL pattern: https://api.dinesync.com/v1/{service}
- All requests require Authorization: Bearer <token> header (except public menu endpoints)
- Tenant identified via X-Restaurant-ID header or subdomain
- JSON request/response bodies throughout
- Standard HTTP status codes: 200, 201, 400, 401, 403, 404, 409, 500

### 8.2 Key Endpoints

#### Auth Service

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | /auth/register | Register new owner account | No |
| POST | /auth/login | Login, receive JWT + refresh token | No |
| POST | /auth/refresh | Refresh access token | Refresh token |
| POST | /auth/logout | Invalidate session | Yes |
| GET | /auth/me | Get current user profile | Yes |

### Menu Service

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | /menu/public/{restaurantId} | Get full menu (public, for QR scan) | No |
| POST | /menu/categories | Create menu category | Owner |
| POST | /menu/dishes | Add new dish | Owner |
| PATCH | /menu/dishes/{id}/availability | Toggle dish availability | Owner/Manager |
| PUT | /menu/dishes/{id} | Update dish details | Owner |
| DELETE | /menu/dishes/{id} | Remove dish | Owner |

### Order Service

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | /orders | Place a new order (from QR table session) | No / Session token |
| GET | /orders/live | Get all live orders (manager view) | Manager |
| PATCH | /orders/{id}/approve | Approve order (send to kitchen) | Manager |
| PATCH | /orders/{id}/serve | Mark order as served | Manager |
| GET | /orders/{id} | Get single order details | Yes |

## 9. Event Architecture (Kafka Topics)

| Topic | Published By | Consumed By | Payload Summary |
|---|---|---|---|
| order.placed | Order Service | Notification Service | order_id, table_id, restaurant_id, items[] |
| order.approved | Order Service | Kitchen Queue Service, Notification Service | order_id, approved_by, timestamp |
| order.status_updated | Kitchen Queue Service | Notification Service, Order Service | order_id, new_status, chef_id |
| order.completed | Order Service | Inventory Service, Payment Service | order_id, items[], total_amount |
| payment.confirmed | Payment Service | Order Service, Notification Service | order_id, payment_id, amount |
| stock.low_alert | Inventory Service | Notification Service | dish_id, current_stock, threshold |
| stock.depleted | Inventory Service | Menu Service, Notification Service | dish_id, restaurant_id |

# 10. Subscription & Monetization

| Feature | Free Trial | Starter (₹999/mo) | Growth (₹2,499/mo) | Enterprise |
|---|---|---|---|---|
| Trial Period | 1 month | - | - | - |
| Max QR Tables | 2 | 10 | 50 | Unlimited |
| Menu Updates/Month | Unlimited (trial) | 10 | Unlimited | Unlimited |
| Staff Roles | 1 (Owner only) | 3 roles | 10 staff logins | Unlimited |
| WhatsApp Alerts | No | No | Yes | Yes |
| Menu Customization | No | Standard | Full | White-label |
| E-Bill per Table | No | Add-on | Included | Included |
| Analytics Dashboard | No | Basic | Advanced | Full + Export |
| POS Integration | No | No | No | Yes |
| API Access | No | No | No | Yes |
| Support | Community | Email | Priority Email | Dedicated Manager |

# 11. Deployment Architecture

## 11.1 AWS Infrastructure

| AWS Service | Usage |
|---|---|
| EC2 (t3.medium) | One instance per microservice (or ECS for containerized deployment) |
| RDS (PostgreSQL) | Managed database per service (or shared RDS with separate schemas in dev) |
| ElastiCache (Redis) | Managed Redis for queue and caching |
| Amazon MSK | Managed Kafka for event streaming |
| S3 | Dish images, QR codes, invoice PDFs |
| ALB (Application Load Balancer) | Load balancing across service instances |
| CloudWatch | Logs, metrics, and alerts |

| AWS Service | Usage |
|---|---|
| Route 53 | Subdomain routing (*.dinesync.com → ALB) |
| ACM (SSL) | HTTPS certificates for all subdomains |

### 11.2 CI/CD Pipeline (GitHub Actions)

- On PR: Run unit tests, lint, and type-check for affected service
- On merge to main: Build Docker image, push to ECR, deploy to EC2/ECS
- Environment-specific configs via GitHub Secrets and .env injection
- Rollback strategy: previous Docker image tag on failed health check
- Slack/email notification on deployment success or failure

### 11.3 Local Development Setup

**Docker Compose**

All 9 services + PostgreSQL instances + Redis + Kafka + Zookeeper run locally via a single docker-compose up command. Each developer gets a fully isolated environment with seeded test data.

## 12. Security Considerations

- JWT access tokens expire in 15 minutes; refresh tokens in 7 days
- All passwords hashed with bcrypt (12 rounds)
- HTTPS enforced across all endpoints via AWS ACM + ALB
- RBAC middleware on every protected route — roles verified server-side
- Tenant isolation: every DB query includes restaurant_id filter
- Input validation via Zod schemas on all incoming request bodies
- SQL injection prevention via Prisma parameterized queries
- Razorpay webhook signature verification to prevent payment fraud
- Rate limiting per IP and per tenant via Redis counters at API Gateway
- S3 bucket private by default — presigned URLs for image access

## 13. Project Milestones & Timeline

| Week | Milestone | Deliverable |
|---|---|---|
| 1–2 | Project setup & Auth | API Gateway + Auth Service fully functional with JWT |

| Week | Milestone | Deliverable |
|---|---|---|
| 3–4 | Restaurant & Menu | Restaurant onboarding, QR generation, menu CRUD |
| 5–6 | Order flow | End-to-end order placement → approval → kitchen queue |
| 7–8 | Real-time & Payments | WebSocket dashboards, Kafka events, Razorpay integration |
| 9–10 | Inventory & Notifications | Stock tracking, low-stock alerts, WhatsApp (Growth) |
| 11 | AWS Deployment | All services live on AWS with CI/CD |
| 12 | Testing & Documentation | Full Swagger docs, Postman collection, README per service |

# 14. Interview Talking Points

### 30-Second Pitch

"I built DineSync— a multi-tenant SaaS backend for QR-based restaurant ordering. It's a microservices architecture with 9 independent services communicating via Kafka, with real-time kitchen queues powered by Redis and Socket.io. The platform supports role-based access for customers, floor managers, chefs, and owners, with Razorpay payment integration and deployment on AWS using Docker and GitHub Actions CI/CD."

## Key Technical Decisions to Discuss

- **Each domain (auth, menu, orders, payments) has independent scaling needs and failure isolation:** Why microservices?
- **Order placement is high-concurrency write — async events decouple services and prevent order loss:** Why Kafka over direct API calls?
- **In-memory sorted sets give O(log N) enqueue/dequeue — critical for real-time chef UX:** Why Redis for kitchen queue?
- **Database-per-service pattern ensures no shared state coupling and independent schema evolution:** Why PostgreSQL per service?
- **Clean isolation per restaurant, easy SSL management, and better branding for restaurant owners:** Why multi-tenancy via subdomain?